DAY-2 [RAG (Retrieval-Augmented Generation)]

Why LLMs Need Memory (And Why Copy-Paste Is Dumb)

🟤 DAY-2 GOAL (LOCK THIS IN)

By the end of Day-2, your brain should **automatically reject** this thought:

❌ *"I'll just paste the full code / document into GPT"*

And replace it with:

✅ *"What context does the model ACTUALLY need right now?"*

This single question is what separates:

- ❌ Prompt monkey
- ✅ GenAI engineer

PART-1 [THE CORE PROBLEM RAG SOLVES (THEORY)]

❌ What Most People Do (Wrong Mental Model)

```
Big document / full codebase
        ↓
       LLM
        ↓
     Answer
```

Looks logical.
Completely broken.

❌ Why This Fails

1️⃣ *Context Window Is Finite*

LLMs **cannot read infinite text**.

Every model has a **hard token limit**.

When input exceeds that limit:

- Older tokens are silently dropped
- You get **no warning**

⚠️ This is dangerous because you don't know **what got removed**.

2️⃣ *LLMs Don't Know They're Missing Info*

When crucial info is missing:

- The model **guesses**
- Guessing = **hallucination**

Important truth:

This is NOT a bug
This is how transformers work

🧠 Mental Model (Burn This In)

LLMs have:

- ❌ No memory
- ❌ No state
- ❌ No awareness of past messages

They ONLY see:

- what you send
- right now
- in this request

That’s it.

So how do we solve this?

👉 **We fake memory**

That fake memory is called **RAG**.

🧠 Mini Cheat Sheet (Part-1)

- LLMs are stateless
- Context ≠ memory
- Missing context → hallucination
- RAG = memory illusion

PART-2 [WHAT RAG ACTUALLY IS?]

First: forget the noise

Ignore:

- LangChain
- Fancy frameworks
- Complicated diagrams
- YouTube buzzwords

They hide the **simple idea**.

What RAG really means (in plain English)

RAG means:

Don’t let the AI answer immediately.
First, give it the right information to read.

That’s it.

RAG flow in very simple steps:

Step 1: User asks a question

→ Someone asks something.

Step 2: Find relevant information

→ The system looks into your data and finds useful parts.

Step 3: Send ONLY that info to the AI

→ The AI sees just the important pieces, not everything.

Step 4: Generate the answer

→ The AI explains what it just read.

That's Rag.

Why this works (simple reason)

Because the AI:

- is **not guessing**
- is **reading first**
- then answering

Why we say “everything else is abstraction”

Frameworks only:

- automate these steps

- hide them behind functions
- make them look complex

But under the hood, they all do **exactly this**.

Key shift in thinking (VERY IMPORTANT)

❌ Generation first (wrong thinking)

> "User asks → AI answers from its brain"

This causes:

- guessing
- hallucinations
- confident nonsense

☑ Retrieval first (correct thinking)

"User asks → system finds data → AI answers using data"

This creates:

- factual answers
- evidence-based replies
- reliable systems

Why "retrieval controls truth"

Because:

- The AI can only answer **from what it sees**
- If it sees correct data → correct answer
- If it sees nothing → it imagines

So the **retrieval step decides**:

- what the AI knows
- what the AI is allowed to say

Mini Cheat Sheet

- RAG = find info, then explain it
- AI should read before talking
- LLM comes last, not first
- Retrieval decides correctness

One sentence to lock it in 🔒

RAG works because it **shows the AI the answer first**,
and then asks it to explain.

Interview Answer:

“What exactly is the RAG pipeline, and where is it implemented?”

What is the RAG pipeline? (Short & clear)

RAG pipeline is a two-stage system:

First retrieve relevant data, then generate an answer using an LLM.

It controls what the LLM is allowed to see before it answers.

Where exactly is the RAG pipeline? (IMPORTANT)

👉 **RAG is NOT one file or one tool**
👉 **RAG is a FLOW across multiple components**

You *point to the pipeline* by pointing to **these stages** 👇

**(IMPORTANT)RAG PIPELINE — WITH TECH STACK

1 Document Source

Tech: Files / PDFs / Code / DB

Explanation:
→ This is the source of truth; LLM never reads it directly.

2 Chunking

Tech: Python logic (custom code)

Explanation:
→ Large documents are split into small meaningful pieces.

3 Embeddings

Tech: Gemini Embedding Model (models/embedding-001)

Explanation:
→ Each chunk is converted into numbers that capture meaning.

4 Vector Database

Tech: FAISS

Explanation:
→ Stores embeddings and allows fast semantic search.

5 Query Embedding

Tech: Gemini Embeddings

Explanation:
→ User question is converted into the same vector format.

6 Semantic Retrieval

Tech: FAISS search (index.search)

Explanation:
→ Finds the most relevant chunks using vector distance.

7 Context Assembly

Tech: Python string formatting

Explanation:
→ Retrieved chunks are combined into a context prompt.

8 Generation

Tech: Gemini 2.5 Flash (LLM)

Explanation:
→ LLM generates an answer using only the retrieved context.

If interviewer asks:

"Where do you point and say *this* is the pipeline?"

"The RAG pipeline starts at **document ingestion**, flows through **chunking, embeddings, vector search**, and ends at **LLM generation with retrieved context**. It's implemented across ingestion scripts, vector database, and the API layer—not in a single function."

In One-line

RAG pipeline = data → vectors → search → context → LLM answer

Extra confidence line

“LLMs don’t store knowledge; the RAG pipeline controls knowledge flow.”

PART-3 [THE 4 CORE COMPONENTS OF RAG (THEORY)]

1️⃣ Documents (Source of Truth)

Examples:

- Codebase
- PDFs
- API docs
- Logs
- DB records

📌 Critical rule:

LLM **never sees these directly**

Why?
Because they’re too large.

Mini Cheat Sheet

- Documents = truth layer
- LLM ≠ database

2️⃣ Chunking (MOST IMPORTANT STEP)

You **never embed full files**.

You **split them**.

Example:

```
utils.py
  ├── helper1()
  ├── helper2()
  └── helper3()
```

Becomes:

- Chunk-1 → helper1
- Chunk-2 → helper2
- Chunk-3 → helper3

❗ Bad chunking = useless RAG

❗ Chunking quality > model choice

Why?
Because retrieval happens **at chunk level**, not file level.

Mini Cheat Sheet

- Chunk size matters
- Logical boundaries matter
- Garbage chunks → garbage answers

3️⃣ Embeddings (Text → Numbers) — Easy Way

Problem:
LLMs and databases **cannot understand raw text meaning**.

Solution:
Convert text into **numbers (vectors)**.

What actually happens

Text sentence
→ Embedding model
→ Vector (list of numbers)

Example:
"def login(user, password)"
→ [0.021, -0.91, 0.332, ...]

These numbers represent **meaning**, not words.

Core idea (VERY IMPORTANT)

- Meaning ≠ exact words
- Same meaning → vectors are **closer**
- Different meaning → vectors are **farther**

This closeness is called **semantic similarity**.

Why embeddings matter in RAG

- We **embed documents**
- We **embed user questions**
- Then we compare vectors to find **most relevant info**

No keywords.
Only meaning.

Mini Cheat Sheet

- Embeddings = meaning in numbers
- Not keyword matching

- Same idea, different words → close vectors
- Distance = relevance

4️⃣ Vector Database (Semantic Search)

First, forget the word “database”

When you hear **Vector Database**, don’t imagine:

- tables
- rows
- SQL

Instead, imagine this 👇

A **smart index** that helps the AI quickly find
the **most relevant pieces of information**.

That’s it.

What does a Vector Database store? (Very simple)

It stores **two things together**:

1. A **vector** (numbers that represent meaning).
2. The **original text chunk** (code, paragraph, sentence).

Example:

Vector: [0.12, -0.44, 0.98, ...]
Text: "def login(username, password): ..."

The vector is for **searching**.
The text is for **reading**.

Why do we even need vectors?

Because computers:

- are bad at understanding meaning in text
- are very good at comparing numbers

So we convert text → numbers (embeddings).

What happens when a question comes in?

Let's say the user asks:

"Why is the login function insecure?"

Now follow this slowly 👇

Step 1: Embed the question

The question is converted into numbers:

```
"Why is the login function insecure?"
→ [0.15, -0.31, 0.87, ...]
```

Now the question is also a **vector**.

Step 2: Find nearest vectors

The Vector DB now asks:

> "Which stored vectors are **closest** to this question vector?"

Closest = **most similar in meaning**

It does **not** look for exact words like:

- "login"
- "insecure"

It looks for **meaning similarity**.

Step 3: Retrieve original text chunks

Once it finds the closest vectors, it returns:

- the **text chunks** linked to those vectors

Example retrieved chunk:

```
def login(username, password):
  hardcoded_password = "admin123"
```

Now the AI finally sees **real evidence**.

Why this is called *semantic* search

Because it searches by:

- **meaning**
- **idea**
- **intent**

Not by exact words.

Difference between keyword search and semantic search

Keyword search (old school)

Search:

> "login insecure"

Will only work if those exact words exist.

If the code says:

```
hardcoded_password = "admin123"
```

Keyword search might **miss it**.

Semantic search (vector DB)

Search:

> “login insecure”

Even if the code never says “insecure”
the vector DB understands:

> “Hardcoded password = security issue”

And retrieves it.

That’s the magic.

Why Vector DB is called a “brain index”

Because it works like your brain.

Your brain does not remember:

- exact sentences
- exact words

It remembers:

- ideas
- meaning
- similarity

Vector DB does the same.

What does “distance = relevance” mean?

Distance = how far two vectors are.

- **Small distance** → very similar meaning
- **Large distance** → not related

So:

Smaller distance = more relevant chunk

That’s how ranking works.

Where does FAISS fit in?

FAISS is just a tool.

It:

- stores vectors
- compares them fast
- runs locally on your machine

No cloud.
No setup drama.
Just speed.

Mini Cheat Sheet

- Vector DB stores meaning + text
- Questions are converted to vectors
- Closest vectors = best answers
- Distance decides relevance
- FAISS = fast local vector search

One-line summary (lock this in)

A vector database helps the AI **find the right page to read** before it starts answering.

PART-4 — END-TO-END RAG FLOW (EASY)

Ingestion Phase (One-Time)

1 Load documents
→ Read your files (code, PDFs, text) into the system.

2 Split into chunks
→ Break big files into small meaningful pieces.

3 Generate embeddings
→ Convert each chunk into numbers that represent its meaning.

4 Store in vector DB
→ Save those numbers so they can be searched later.

Query Phase (Every Question)

5 User asks question
→ User sends a question to your system.

6 Embed question
→ Convert the question into numbers (same format as chunks).

7 Retrieve top-k chunks
→ Find the most relevant pieces related to the question.

8 Send chunks + question to LLM
→ Give the AI only the useful information and the question.

9 Generate grounded answer
→ AI answers using facts, not guessing.

One-line Final Summary 🔒

RAG first **finds the right information**, then **asks the AI to explain it**.

Mini Cheat Sheet

- Ingest once
- Query many times
- Never re-embed docs on every question

PART-5 [WHY RAG REDUCES HALLUCINATIONS]

First, what is “hallucination” in simple words?

Hallucination means:

The AI **sounds confident**,
but the answer is **not based on your data**.

It is **not lying on purpose**.
It is **guessing** when it doesn’t have enough information.

What happens WITHOUT RAG (step by step)

Imagine this situation.

You ask the AI:

> “Why is the login function insecure?”

But you **did NOT give**:

- the code
- the file

- the project details

So the AI's brain works like this 👇

> "I cannot see the actual code.
> But in many projects, login functions are insecure because of common reasons.
> I will answer based on what usually happens."

So it replies:

> "This function is probably insecure because it might not hash passwords or may have weak authentication."

Why this answer is dangerous

- It **sounds correct**
- It **sounds confident**
- But it is **not based on YOUR code**

The AI is **filling missing information with assumptions**.

This is what we mean by:

❌ **"This function probably does X..."**

The word **"probably"** is the red flag 🚩
It means **guessing**, not knowing.

Important truth (very important)

LLMs **hate saying**:

> "I don't know."

So when they don't see data, they:

- use general knowledge
- make assumptions
- answer confidently

This confident guessing is called **hallucination**.

Now, what changes WITH RAG?

Now imagine the same question again:

> "Why is the login function insecure?"

But this time, RAG does something **before** the AI answers.

What RAG does first

RAG:

1. Searches your project
2. Finds the **exact login function**
3. Sends that **code snippet** to the AI

Example context sent to AI:

```
def login(username, password):
  hardcoded_password = "admin123"
  if password == hardcoded_password:
    return "Login successful"
```

Now the AI's thinking changes 👇

> "I can SEE the code.
> I don't need to guess.
> I will answer using exactly what is written here."

So the AI answers:

> "This login function is insecure because it uses a hardcoded password and compares passwords in plaintext."

Why this answer is better

Because:

- It points to **real lines of code**

- Anyone can verify it
- No guessing involved

This is what we mean by:

✅ **“Based on this code snippet: def login(...) ...”**

The core reason RAG reduces hallucinations

Very simple reason:

RAG **removes guessing**.

Without RAG:

- Missing info → AI imagines

With RAG:

- Info is present → AI explains

One-line difference (remember this)

❌ Without RAG:

“I don’t see your data, so I’ll answer based on experience.”

✅ With RAG:

“I see your data, so I’ll answer based on facts.”

Real-life analogy (super easy)

Without RAG:

Teacher asks a question. Student **does not have the textbook**. Student answers from memory and guesses.

With RAG:

Student **has the textbook open**. Student reads and answers.

Which answer will be correct?

👉 The second one.

Why we say “RAG forces evidence”

Because RAG:

- forces the AI to **look at proof**
- forces answers to be **grounded**
- does not allow “vibe-based” answers

No proof → imagination
Proof present → truth

Mini Cheat Sheet

- Hallucination = confident guessing
- No RAG = AI guesses
- RAG = AI reads your data
- Reading data = correct answers
- Evidence beats confidence

Final sentence to lock it in 🔒

RAG works because it **shows the AI the answer first**,
and only then asks it to explain.

PART-6 — TECH STACK (BEGINNER-SAFE)

We keep it boring and correct.

Backend

- FastAPI

Embeddings

- Gemini Embeddings

Vector DB

- FAISS (local)

📌 No Pinecone
📌 No LangChain

Fundamentals first.

PART-7 — PROJECT STRUCTURE (VERY IMPORTANT)

```
rag_app/
│
├── main.py          # API layer (query time)
├── ingest.py        # chunk + embed (one-time)
├── vector_store.py  # FAISS logic
├── documents/
│   └── utils.py     # sample source document
│
├── .env             # API keys
└── requirements.txt
```

Key idea:

- Ingest ≠ Query
- Clean separation of concerns

Mini Cheat Sheet

- One responsibility per file
- RAG systems are pipelines

FastAPI

▧ PART-8 — PRACTICAL IMPLEMENTATION (REWRITTEN CLEAN)

We start **from scratch**.
Assume the student knows **nothing**.

🧠 FINAL STACK (LOCK THIS)

Layer	Tool
Embeddings	SentenceTransformers (local)
Vector Search	FAISS
LLM	Gemini 2.5 Flash
API	FastAPI

🗀 PROJECT STRUCTURE (DO NOT CHANGE)

```
day2-rag/
│
├── main.py
```

```
├── ingest.py
├── vector_store.py
├── documents/
│   └── utils.py
│
├── requirements.txt
├── .env
└── README.md   (optional)
```

FILE 1 — requirements.txt

```
fastapi
uvicorn
faiss-cpu
numpy
python-dotenv
sentence-transformers
google-generativeai
```

Why each one exists

- sentence-transformers → **local embeddings**
- faiss-cpu → similarity search
- google-generativeai → Gemini 2.5 Flash
- No OpenAI. No quotas. No drama.

FILE 2 — .env

```
GEMINI_API_KEY=your_real_gemini_api_key_here
```

⚠️ Never commit this file.

📄 FILE 3 — documents/utils.py (SOURCE OF TRUTH)

```
def login(username, password):
    hardcoded_password = "admin123"
    if password == hardcoded_password:
        return "Login successful"
    return "Login failed"


def helper_function():
    print("Just a helper")
```

📌 This file is what RAG learns from.

📌 If it’s not here, the LLM must say “Not found”.

📄 FILE 4 — ingest.py (CHUNK + EMBED)

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Load once (important)
model = SentenceTransformer("all-MiniLM-L6-v2")


def chunk_text(text, chunk_size=200):
    words = text.split()
    chunks = []

    for i in range(0, len(words), chunk_size):
        chunk = " ".join(words[i:i + chunk_size])
```

```
        chunks.append(chunk)

    return chunks


def load_and_embed_documents():
    with open("documents/utils.py", "r") as f:
        text = f.read()

    chunks = chunk_text(text)

    embeddings = model.encode(
        chunks,
        convert_to_numpy=True,
        normalize_embeddings=True
    )

    return chunks, embeddings.astype("float32")
```

Key points (READ CAREFULLY)

- Embeddings are **local**
- No API calls
- No rate limits
- normalize_embeddings=True → better search

FILE 5 — vector_store.py (FAISS)

```
import faiss


def build_index(vectors):
    dimension = vectors.shape[1]
    index = faiss.IndexFlatL2(dimension)
    index.add(vectors)
```

```
    return index
```

Simple. Stable. Fast.

FILE 6 — main.py (RAG API)

```
import os
import numpy as np
import google.generativeai as genai
from fastapi import FastAPI
from dotenv import load_dotenv

from ingest import load_and_embed_documents
from vector_store import build_index

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

app = FastAPI()

# Ingest at startup
chunks, vectors = load_and_embed_documents()
index = build_index(vectors)


@app.post("/ask")
def ask(question: str):
    from sentence_transformers import SentenceTransformer

    embedder = SentenceTransformer("all-MiniLM-L6-v2")
    query_vector = embedder.encode(
        question,
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")
```

```
    _, indices = index.search(
        np.array([query_vector]),
        k=3
    )

    context = "\n".join([chunks[i] for i in indices[0]])

    model = genai.GenerativeModel("gemini-2.5-flash")

    response = model.generate_content(
        f"""
Answer ONLY using the context below.
If not present, say "Not found in context".

Context:
{context}

Question:
{question}
"""
    )

    return {"answer": response.text}
```

▶ HOW TO RUN (NO CONFUSION)

1 Create venv

```
python3 -m venv venv
source venv/bin/activate
```

2️⃣ Install deps

pip install -r requirements.txt

3️⃣ Start server

python3 -m uvicorn main:app --reload

You should see:

Uvicorn running on http://127.0.0.1:8000

🧪 HOW TO TEST (STEP-BY-STEP)

Open:

http://127.0.0.1:8000/docs

☑️ Test Case 1 — Security Question

```
{
  "question": "Why is the login function insecure?"
}
```

Expected:

- Mentions **hardcoded password**
- Quotes logic from utils.py

❌ Test Case 2 — Non-existent Info

```
{
  "question": "Does this app use OAuth?"
}
```

Expected:

"Not found in context"

This proves RAG is working.

🧠 MINI CHEAT SHEET (BURN THIS)

- Embeddings ≠ LLM
- Retrieval happens BEFORE generation
- Local embeddings = stable systems
- LLM is last step, not first
- If context is wrong → answer is wrong

🔥 FINAL DAY-2 TRUTH

LLMs reason.
Embeddings remember.
FAISS retrieves.
RAG makes them cooperate.

You're no longer "using AI".
You're **engineering GenAI systems** now.

ERROR HANDLING

🟤 What this message actually means

FutureWarning:
All support for the `google.generativeai` package has ended.
Please switch to the `google.genai` package

Translation (simple words):

- google.generativeai 👉 **deprecated**
- It still works **today**
- It may break **tomorrow**
- Google has moved everything to **google.genai**

So your code runs **now**, but:

❌ keeping this package = technical debt
✅ switching now = professional move

🧠 Important clarification (so you don't panic)

- ❌ This is **NOT** why your app fails
- ❌ This is **NOT** a runtime crash
- ✅ Your RAG logic is correct
- ✅ Your SentenceTransformer setup is correct

This is **just a library migration warning**.

☑ THE RIGHT FIX (DO THIS ONCE)

We'll:

1. Replace the deprecated package
2. Update **imports**
3. Update **Gemini calls**
4. Keep **Gemini 2.5 Flash**

🛠 STEP-BY-STEP FIX (ZERO CONFUSION)

1 Update requirements.txt

❌ REMOVE this line

google-generativeai

☑ ADD this instead

google-genai

So final snippet:

fastapi
uvicorn
faiss-cpu
numpy
python-dotenv
sentence-transformers
google-genai

2️⃣ Reinstall deps (important)

pip uninstall google-generativeai -y
pip install -r requirements.txt

3️⃣ Fix main.py (THIS IS THE REAL CHANGE)

❌ OLD (deprecated)

import google.generativeai as genai
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

☑️ NEW (correct, supported)

from google import genai

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

4️⃣ Fix the Gemini call (VERY IMPORTANT)

❌ OLD

model = genai.GenerativeModel("gemini-2.5-flash")
response = model.generate_content(prompt)
text = response.text

☑ NEW (google.genai syntax)

```
response = client.models.generate_content(
   model="gemini-2.5-flash",
   contents=prompt
)

text = response.text
```

5 FINAL main.py (CLEAN & FUTURE-SAFE)

```
import os
import numpy as np
from fastapi import FastAPI
from dotenv import load_dotenv
from google import genai
from sentence_transformers import SentenceTransformer

from ingest import load_and_embed_documents
from vector_store import build_index

load_dotenv()

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

app = FastAPI()

# Load RAG components at startup
chunks, vectors = load_and_embed_documents()
index = build_index(vectors)

embedder = SentenceTransformer("all-MiniLM-L6-v2")
```

```
@app.post("/ask")
def ask(question: str):
    query_vector = embedder.encode(
        question,
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    _, indices = index.search(
        np.array([query_vector]),
        k=3
    )

    context = "\n".join([chunks[i] for i in indices[0]])

    prompt = f"""
Answer ONLY using the context below.
If not present, say "Not found in context".

Context:
{context}

Question:
{question}
"""

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    return {"answer": response.text}
```

▶️ Run again

python3 -m uvicorn main:app --reload

☑️ Expected result

- ❌ No FutureWarning
- ✅ Server starts clean
- ✅ /docs works
- ✅ RAG answers grounded

🧠 WHY THIS MATTERS (ENGINEER MINDSET)

You just learned:

- Libraries deprecate **fast**
- Warnings are **early signals**
- Fixing infra early = senior behavior
- Local embeddings + hosted LLM = best practice

This is **exactly how real GenAI systems are maintained**.